

Lesson 13 — Principal Component Analysis (PCA)

We're moving directly after Lesson 12: K-Means Clustering.

1. What is PCA?

PCA (Principal Component Analysis) is a dimensionality reduction technique.

It takes a dataset with many features and converts them into a smaller number of new features called Principal Components, while trying to preserve as much important information (variance) as possible.

Example

Suppose you have:

```
10 features
  ↓
PCA
  ↓
2 principal components
```

Instead of working with 10 dimensions, you can visualize or train a model using just 2.

2. Why do we need PCA?

Imagine a dataset with:

100 features

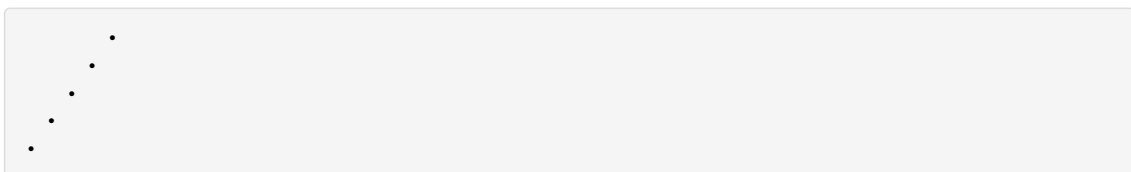
Problems:

- More computation
- Harder to visualize
- Some features may contain redundant information
- High-dimensional data can make ML models harder to work with

PCA helps by compressing the information.

3. Simple Intuition

Imagine these points:



The data is mainly spreading in one direction.

PCA tries to find the direction containing the maximum variance.

That direction becomes:

Principal Component 1



Then PCA finds another direction that captures the next highest amount of variance while being perpendicular to the first.

4. What is Variance?

Variance tells us how much the data is spread out.

For PCA:

More variance generally means more information about the structure of the data.

PCA tries to preserve the directions with the most variance.

5. Principal Components

Suppose your original dataset has:

Feature 1 Feature 2 Feature 3 Feature 4 Feature 5

PCA can transform them into:

PC1 PC2 PC3

where:

- PC1 → captures the most variance
- PC2 → captures the next most variance
- PC3 → captures the next most variance

And so on.

6. Important: Scaling Before PCA

Just like K-Means, PCA is affected by feature scales.

For example:

```
Age      → 18-80
Salary   → 20,000-2,000,000
```

Salary could dominate the analysis.

So we generally standardize first:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

7. PCA in Scikit-Learn

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)
```

Now:

```
Original:
X → many features

After PCA:
X_pca → 2 features
```

8. n_components

This determines how many principal components you want.

```
PCA(n_components=2)
```

means:

Reduce the data to 2 dimensions.

For example:

```
PCA(n_components=3)
```

means:

Keep 3 principal components.

9. Explained Variance Ratio ☆

This is one of the most important concepts in PCA.

```
print(pca.explained_variance_ratio_)
```

Example:

```
[0.65 0.25]
```

This means:

PC1 → 65% of variance PC2 → 25% of variance

Together:

65% + 25% = 90%

So our 2 components preserve approximately 90% of the variance.

10. Choosing the Number of Components

Instead of randomly choosing 2 or 3 components, we can examine cumulative explained variance.

```
from sklearn.decomposition import PCA

pca = PCA()

X_pca = pca.fit_transform(X_scaled)

print(pca.explained_variance_ratio_)
```

Then:

```
import numpy as np

cumulative_variance = np.cumsum(
    pca.explained_variance_ratio_
)

print(cumulative_variance)
```

Example:

```
PC1      → 50%
PC1+PC2  → 75%
PC1+PC2+PC3 → 89%
PC1...PC4 → 95%
```

You might choose 4 components if you want to retain about 95% of the variance.

11. PCA Visualization

One major advantage of PCA is visualization.

Suppose your dataset has:

20 features

You can't directly plot 20 dimensions.

We can reduce it:

```
pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)
```

Then:

```
import matplotlib.pyplot as plt

plt.scatter(X_pca[:, 0], X_pca[:, 1])

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")

plt.show()
```

Now a high-dimensional dataset can be visualized in 2D.

12. Important PCA Attributes

components_

```
print(pca.components_)
```

Shows the directions of the principal components and how original features contribute to them.

explained_variance_ratio_

```
print(pca.explained_variance_ratio_)
```

Shows how much variance each component explains.

n_components_

```
print(pca.n_components_)
```

Shows the number of components retained.

13. PCA vs K-Means

These two are very different.

K-Means	PCA
Clustering algorithm	Dimensionality reduction
Unsupervised	Unsupervised
Creates groups	Creates new features
Needs n_clusters	Uses n_components
Uses distance	Uses variance/directions

They can also be used together:

```
Original Data
↓
StandardScaler
↓
PCA
↓
Reduced Features
↓
K-Means
↓
Clusters
```

This is common when the original dataset has many features.

14. PCA in SentinelAI

PCA can eventually be useful in your SentinelAI project.

Your telemetry may contain many features:

CPU Memory Latency Error Rate ...

As the system grows, you could have many telemetry-derived features.

PCA could help:

```
Many telemetry features
  ↓
PCA
  ↓
Important components
  ↓
Visualization / analysis
```

For example, PCA could help visualize whether different service states naturally form different regions in feature space.

But PCA does not itself predict failures. It is mainly for dimensionality reduction and analysis.

🧠 Lesson 13 — What You Must Remember

PCA = Dimensionality Reduction

```
Many features
  ↓
PCA
  ↓
Fewer features
```

Core ideas:

- PCA finds directions of maximum variance.
- Those directions are called principal components.
- PC1 captures the most variance.
- PC2 captures the next most.
- Scaling is generally important before PCA.
- `n_components` controls how many components are kept.
- `explained_variance_ratio_` tells how much variance each component retains.
- PCA can reduce high-dimensional data to 2D/3D for visualization.
- PCA is not a prediction algorithm.
- PCA can be combined with other ML algorithms.

One-line definition:

PCA is an unsupervised dimensionality-reduction technique that transforms many correlated features into fewer principal components while preserving as much variance as possible.

```
# =====
# LESSON 13 – PCA (PRINCIPAL COMPONENT ANALYSIS)
# Complete Example
# =====

# 1. IMPORT LIBRARIES
# -----

import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

```

# =====
# 2. LOAD DATASET
# =====

wine = load_wine()

X = wine.data
y = wine.target

print("Original shape of X:", X.shape)
print("Target shape:", y.shape)

# =====
# 3. FEATURE NAMES
# =====

print("\nFeatures:")
print(wine.feature_names)

# =====
# 4. FEATURE SCALING
# =====
# PCA is affected by feature scale.
# So we standardize the features first.

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

print("\nScaled data shape:", X_scaled.shape)

# =====
# 5. APPLY PCA WITH 2 COMPONENTS
# =====
# Original dataset has 13 features.
# We reduce it to only 2 features.

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)

print("\nPCA data shape:", X_pca.shape)

# =====
# 6. VIEW THE TRANSFORMED DATA
# =====

print("\nFirst 5 PCA rows:")
print(X_pca[:5])

# =====
# 7. EXPLAINED VARIANCE RATIO
# =====
# Shows how much variance each principal component explains.

print("\nExplained Variance Ratio:")
print(pca.explained_variance_ratio_)

# =====
# 8. TOTAL VARIANCE RETAINED
# =====

total_variance = pca.explained_variance_ratio_.sum()

```

```

print("\nTotal variance retained:")
print(total_variance)

print("\nTotal variance retained (%):")
print(total_variance * 100)

# =====
# 9. PRINCIPAL COMPONENTS
# =====
# Shows how original features contribute to each component.

print("\nPrincipal Components:")
print(pca.components_)

# =====
# 10. COMPONENT SHAPE
# =====

print("\nShape of principal components:")
print(pca.components_.shape)

# =====
# 11. VISUALIZE PCA RESULT
# =====

plt.figure(figsize=(8, 6))

plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=y
)

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")

plt.title("Wine Dataset after PCA")

plt.colorbar(label="Wine Class")

plt.show()

# =====
# 12. CUMULATIVE EXPLAINED VARIANCE
# =====
# To decide how many components we should keep,
# calculate PCA using ALL components.

pca_full = PCA()

X_pca_full = pca_full.fit_transform(X_scaled)

explained_variance = pca_full.explained_variance_ratio_

cumulative_variance = np.cumsum(explained_variance)

print("\nExplained variance of all components:")
print(explained_variance)

print("\nCumulative explained variance:")
print(cumulative_variance)

# =====
# 13. PLOT CUMULATIVE EXPLAINED VARIANCE
# =====

plt.figure(figsize=(8, 6))

```



```

plt.plot(
    range(1, len(cumulative_variance) + 1),
    cumulative_variance,
    marker="o"
)

plt.xlabel("Number of Principal Components")
plt.ylabel("Cumulative Explained Variance")

plt.title("PCA - Cumulative Explained Variance")

plt.grid()

plt.show()

# =====
# 14. FIND NUMBER OF COMPONENTS FOR 95% VARIANCE
# =====

components_95 = np.argmax(cumulative_variance >= 0.95) + 1

print("\nComponents required to retain at least 95% variance:")
print(components_95)

# =====
# 15. PCA USING 95% VARIANCE
# =====
# Instead of manually choosing the number,
# PCA can automatically keep enough components
# to preserve 95% variance.

pca_95 = PCA(n_components=0.95)

X_pca_95 = pca_95.fit_transform(X_scaled)

print("\nOriginal shape:")
print(X.shape)

print("\nShape after PCA with 95% variance:")
print(X_pca_95.shape)

print("\nActual number of components:")
print(pca_95.n_components_)

print("\nVariance retained:")
print(pca_95.explained_variance_ratio_.sum())

# =====
# 16. PCA COMPONENTS
# =====

print("\nPCA Components:")
print(pca_95.components_)

# =====
# 17. RECONSTRUCTION
# =====
# PCA can approximately reconstruct the original data
# from the reduced representation.

pca_reconstruction = PCA(n_components=2)

X_reduced = pca_reconstruction.fit_transform(X_scaled)

X_reconstructed = pca_reconstruction.inverse_transform(X_reduced)

print("\nOriginal scaled data shape:")

```

```

print(X_scaled.shape)

print("\nReduced data shape:")
print(X_reduced.shape)

print("\nReconstructed data shape:")
print(X_reconstructed.shape)

# =====
# 18. COMPARE ORIGINAL AND RECONSTRUCTED DATA
# =====

print("\nOriginal first row:")
print(X_scaled[0])

print("\nReconstructed first row:")
print(X_reconstructed[0])

# =====
# 19. COMPLETE PCA PIPELINE
# =====
# The basic workflow you should remember:

print("""
=====
PCA WORKFLOW
=====

Original Data
  |
  v
StandardScaler
  |
  v
Scaled Data
  |
  v
PCA
  |
  v
Reduced Dimensions
  |
  v
Visualization / ML Model

=====
""")

```